

Projet Machine Learning

Prédiction des Hits sur Spotify

Clémence Jin

M1 IDD — 2025–2026

Table des matières

1	Introduction	3
2	Dataset	3
2.1	Source et description	3
2.2	Sélection des variables (Feature Selection)	4
2.3	Nettoyage et dédoublement	4
3	Analyse exploratoire (EDA)	4
3.1	Traitement des valeurs manquantes	4
3.2	Distribution de la popularité	5
3.3	Création de la variable cible binaire	5
3.4	Diagnostic du déséquilibre des classes	6
3.5	Analyse univariée des caractéristiques audio (features)	6
3.6	Corrélations avec la popularité	7
4	Préparation des données	9
4.1	Encodage des variables catégorielles	9
4.2	Transformations log	9
4.3	Séparation train / test	9
4.4	Gestion du déséquilibre des classes	10
5	Méthodes de machine learning	10
5.1	Régression logistique (baseline)	10
5.2	k plus proches voisins (k-NN)	10
5.3	Naive Bayes	11
5.4	Arbre de décision	11
5.5	Random Forest	12
5.6	Gradient Boosting (HistGradientBoosting)	13
5.7	Balanced Random Forest (Rééchantillonnage bootstrap)	13
5.8	Sous-échantillonnage aléatoire (Random Undersampling)	13
5.9	Sur-échantillonnage synthétique (SMOTE)	14

6	Comparaison des méthodes	14
6.1	Tableau récapitulatif	14
6.2	Analyse des résultats	14
6.3	Performance globale et stabilité (CV)	15
6.4	Capacité de discrimination (Courbes ROC)	16
6.5	Analyse matrice de confusion et Precision/Recall	17
6.6	Sélection du modèle final : le duel RF vs Balanced RF	18
6.7	Évaluation finale sur le Test Set	18
7	Discussion	19
7.1	Borne de performance et état de l’art	19
7.2	Impact des choix de preprocessing	20
7.3	Perspectives	20
8	Conclusion	21

1 Introduction

La prédiction du succès commercial d’une chanson est un problème ouvert depuis les années 2000, désigné dans la littérature sous le terme de *Hit Song Science* [2]. L’avènement des plateformes de streaming a rendu ce problème à la fois plus mesurable (via des scores de popularité continus) et plus complexe, car le succès d’une chanson dépend de facteurs audio intrinsèques, mais aussi de facteurs exogènes difficilement quantifiables (artiste, label, date de sortie, promotion).

L’objectif de ce projet est de prédire, à partir des caractéristiques audio extraites par l’API Spotify, si une chanson sera un *hit* ou un *non-hit* (tâche de classification binaire supervisée). Ce cadre permet de comparer plusieurs familles de méthodes de machine learning, d’explorer les défis liés aux données déséquilibrées, et de s’interroger sur les limites fondamentales d’un tel problème.

2 Dataset

2.1 Source et description

Nous utilisons le **Spotify Tracks Dataset** disponible publiquement sur Kaggle [1], collecté via l’API Web officielle de Spotify. Le fichier brut contient **114 000 lignes** et **21 colonnes**, chaque ligne correspondant à une chanson dans un genre musical donné.

Les variables disponibles sont les suivantes :

Variable	Type	Description
popularity	int [0–100]	Score de popularité Spotify (cible brute)
Unnamed: 0	int	Index de ligne
track_id	object	ID du morceau
artists	object	Nom de l’artiste
album_name	object	Nom de l’album
track_name	object	Nom du morceau
danceability	float [0–1]	Aptitude à la danse
energy	float [0–1]	Intensité perceptuelle
loudness	float (dB)	Volume sonore global
speechiness	float [0–1]	Présence de paroles parlées
acousticness	float [0–1]	Caractère acoustique
instrumentalness	float [0–1]	Absence de voix
liveness	float [0–1]	Enregistrement en live
valence	float [0–1]	Positivité musicale
tempo	float (BPM)	Tempo
duration_ms	int	Durée en millisecondes
explicit	bool	Contenu explicite
key	int [0–11]	Tonalité musicale
mode	int {0,1}	Majeur (1) ou mineur (0)
time_signature	int	Signature rythmique
track_genre	str	Genre musical (114 valeurs)

TABLE 1 – Variables du dataset Spotify (sélection des features utilisées).

2.2 Sélection des variables (Feature Selection)

Sur les 21 colonnes initiales, nous avons opéré une sélection stricte pour garantir la pertinence du modèle :

- **Exclusion des identifiants** : Les variables `track_id`, `track_name` et `album_name` ont été écartées en raison de leur trop grande cardinalité. Leur inclusion risquerait de provoquer un sur-apprentissage (overfitting) sans apporter de valeur prédictive généralisable.
- **Suppression des biais techniques** : La colonne `Unnamed: 0`, simple index de ligne, a été supprimée pour éviter toute fuite de données (*data leakage*) liée à l'ordre d'enregistrement des observations.
- **Focus sur l'audio** : Bien que le nom de l'artiste (`artists`) soit un facteur de succès majeur, nous avons choisi de l'exclure afin de concentrer l'analyse sur la capacité des caractéristiques purement musicales et acoustiques (fournies par Spotify) à expliquer la popularité d'un titre.
- **Analyse de la variance et du bruit** : Toutes les caractéristiques acoustiques présentent une variance non nulle, ce qui signifie qu'elles portent toutes une information potentiellement discriminante

2.3 Nettoyage et dédoublement

Le dataset présente une particularité structurelle importante : une même chanson peut apparaître plusieurs fois si elle appartient à plusieurs genres. On dénombre ainsi **24 259 doublons** sur l'identifiant unique `track_id`. Afin d'éviter toute fuite d'information entre train et test (*data leakage*), nous conservons pour chaque chanson uniquement la ligne correspondant au score de popularité le plus élevé, puis effectuons le split train/test. Après dédoublement, le dataset comporte **89 741 chansons uniques**.

Les valeurs manquantes sont quasiment absentes des variables numériques (3 NaN sur les colonnes textuelles `artists`, `album_name` et `track_name`, qui ne sont pas utilisées comme features). Aucune imputation n'est donc nécessaire.

3 Analyse exploratoire (EDA)

3.1 Traitement des valeurs manquantes

Avant toute analyse, nous avons audité la qualité des données. Le dataset est d'une grande fiabilité technique : sur les 114 000 lignes initiales, seules 3 observations présentent des valeurs manquantes (NaN), localisées exclusivement sur les colonnes textuelles (`artists`, `album_name`, `track_name`).

- **Proportion de NaN** : $< 0,001\%$ pour les colonnes concernées.
- **Variables numériques** : 0% de valeurs manquantes sur l'ensemble des caractéristiques audio (`danceability`, `energy`, etc.).

Compte tenu de cette proportion insignifiante et du fait que les colonnes textuelles sont écartées de la modélisation, aucune stratégie d'imputation complexe n'a été nécessaire.

3.2 Distribution de la popularité

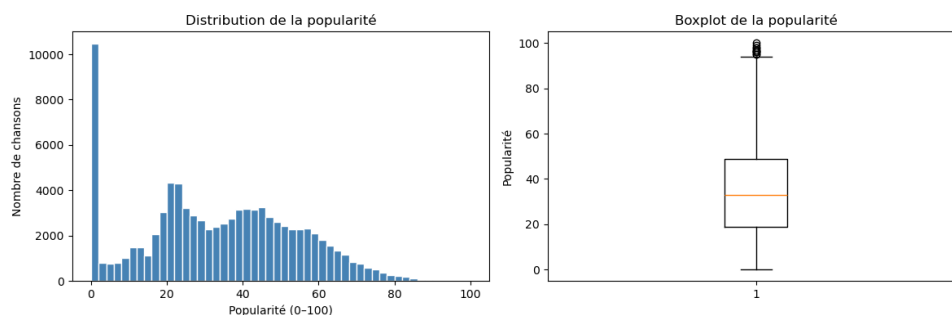


FIGURE 1 – Distribution (histogramme) et Boxplot de la variable Popularité.

La variable `popularity` est distribuée de manière quasi-symétrique autour de sa médiane :

$$\mu = 33,21, \quad \sigma^2 = 423,30, \quad \text{médiane} = 33,0.$$

La distribution est légèrement étalée vers les valeurs élevées (skew positif modéré), avec un pic marqué autour de 0 correspondant aux chansons peu ou pas écoutées.

3.3 Création de la variable cible binaire

Nous définissons un *hit* comme une chanson dont le score de popularité est supérieur ou égal au **75^e percentile**, soit un seuil de **49**. Ce choix est motivé par :

- son caractère objectif et reproductible (percentile de la distribution empirique) ;
- le fait qu'il sépare les chansons populaires (top 25%) du reste.

La répartition résultante est la suivante :

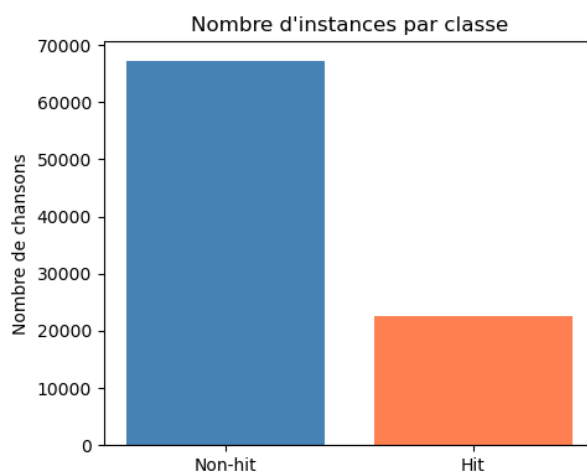


FIGURE 2 – Répartition des classes après binarisation (Seuil = 49).

Non-hit (0) : 67 220 chansons (74,9%), Hit (1) : 22 521 chansons (25,1%).

3.4 Diagnostic du déséquilibre des classes

Ce déséquilibre 75/25 est un point critique de la modélisation. Un classifieur trivial qui prédirait systématiquement **Non-hit** obtiendrait une accuracy de **74,9%** sans avoir rien appris. L'accuracy est donc une métrique trompeuse dans ce contexte, et nous adoptons le **F1-score** comme métrique principale pour toutes les évaluations et sélections d'hyperparamètres.

3.5 Analyse univariée des caractéristiques audio (features)

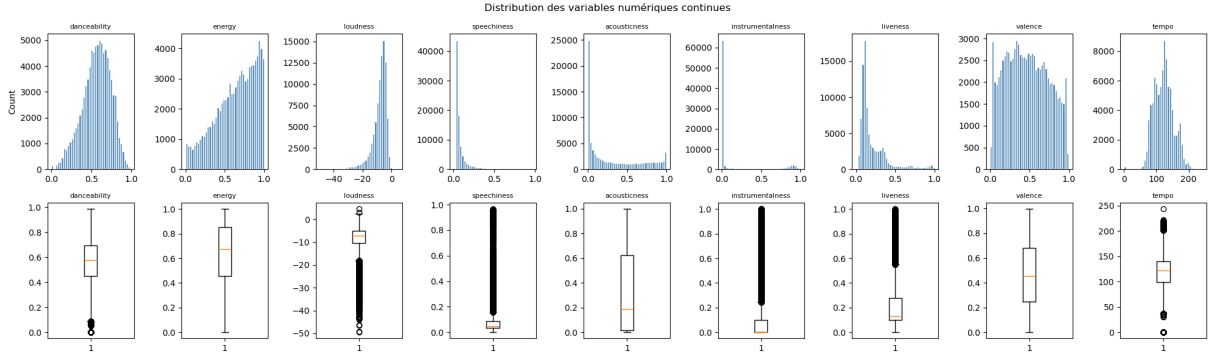


FIGURE 3 – Distributions et Boxplots des variables numériques continues.

Variable	Moyenne	Variance
danceability	0.5622	0.0312
energy	0.6345	0.0658
loudness	−8.4990	27.2640
speechiness	0.0874	0.0128
acousticness	0.3283	0.1145
instrumentalness	0.1734	0.1049
liveness	0.2170	0.0380
valence	0.4695	0.0691
tempo	122.06	907.07

TABLE 2 – Statistiques descriptives des variables numériques continues.

L'examen des distributions (Fig. 3) révèle des profils variés :

- **Variables quasi-normales** : **danceability**, **loudness**, **valence**... présentent des formes en cloche exploitables par la plupart des modèles.
- **Asymétrie forte** : **instrumentalness** présente une distribution fortement asymétrique (skewed) : sa moyenne est 0.17 mais sa médiane avoisine 0.00006, indiquant que la quasi-totalité des chansons comportent des voix, avec une longue queue vers 1 pour les instrumentales pures. La même observation s'applique dans une moindre mesure à **acousticness** et **liveness**. Ces distributions appellent des transformations (voir Section 4).

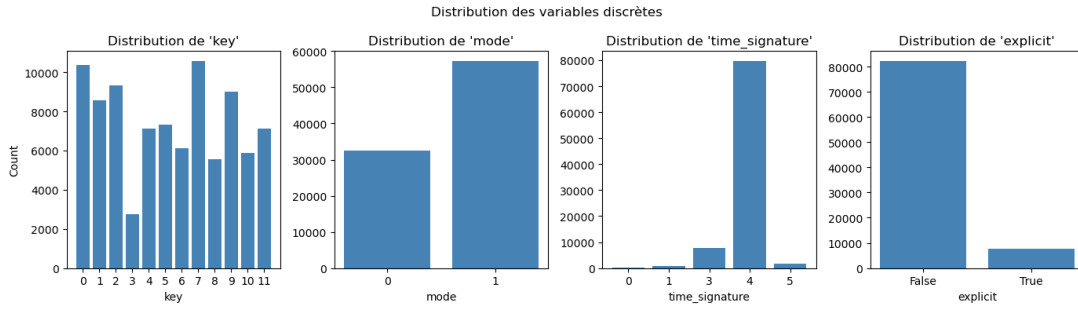


FIGURE 4 – Distribution des variables discrètes et catégorielles.

Concernant les variables discrètes (Fig. 4), on note une prédominance du **mode** majeur (1) et de la signature rythmique en 4/4 (**time_signature** = 4). La variable **explicit** confirme que seul un petit segment du dataset contient du contenu explicite.

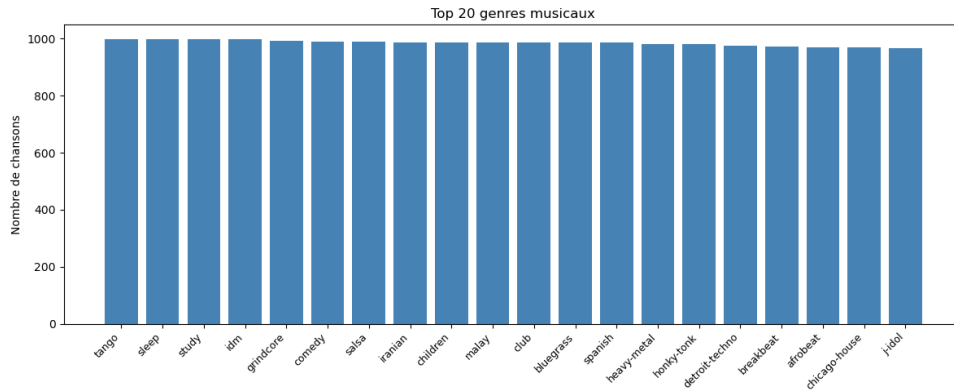


FIGURE 5 – TOP 20 des genres musicaux.

Le dataset couvre 114 genres de manière très équilibrée (environ 1000 titres par genre, Fig. 5), ce qui garantit que le modèle ne sera pas biaisé en faveur d'un style musical particulier au détriment des autres.

3.6 Corrélations avec la popularité

Les corrélations de Pearson entre les features numériques et **popularity** sont toutes très faibles :

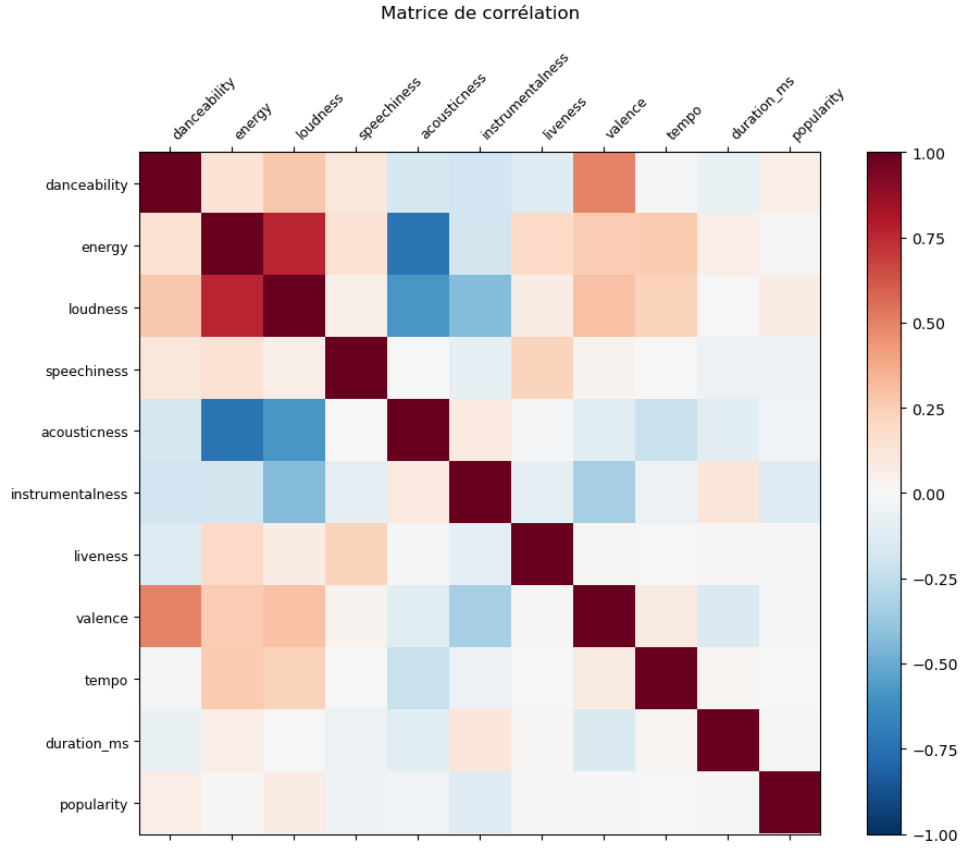


FIGURE 6 – Matrice de corrélation de Pearson entre les variables audio.

Feature	Corrélation avec popularity
loudness	+0.072
danceability	+0.064
energy	+0.014
tempo	+0.007
valence	−0.012
liveness	−0.014
duration_ms	−0.023
acousticness	−0.039
speechiness	−0.047
instrumentalness	−0.128

TABLE 3 – Corrélations de Pearson entre features audio et popularité.

Ces corrélations faibles constituent une **limite fondamentale** du dataset : les caractéristiques audio intrinsèques d’une chanson n’expliquent qu’une très petite fraction de sa popularité. Ce résultat est cohérent avec la littérature sur la *Hit Song Science* [2, 3], qui souligne l’importance des facteurs exogènes (artiste, promotion, réseau social). Le F1-score attendu est donc structuellement limité, indépendamment des modèles utilisés.

4 Préparation des données

4.1 Encodage des variables catégorielles

Variable explicit. Convertie de booléen en entier (0/1).

Variable track_genre — One-Hot Encoding. Dans une première version du pipeline, nous avons utilisé un `LabelEncoder` de scikit-learn, qui assigne un entier arbitraire à chaque genre (`acoustic= 0`, `blues= 1`, ...). Ce choix introduit une relation ordinale fictive entre les genres (le modèle interprète alors `jazz > classical > acoustic` numériquement), ce qui n'a aucun sens sémantique et dégrade les performances.

Nous avons corrigé ce problème en utilisant un **One-Hot Encoder** (`OneHotEncoder` de scikit-learn), qui crée une colonne binaire indépendante pour chacun des 114 genres. Le dataset final comprend ainsi $14 + 114 = \mathbf{128}$ features. La matrice de features est stockée au format *sparse* pour limiter l'empreinte mémoire.

Avant l'adoption du One-Hot Encoding, une première approche utilisant le `LabelEncoder` a été testée pour traiter la variable `track_genre`.

- **Approche initiale :** Le `LabelEncoder` affecte un entier unique ($0, 1, 2, \dots, 113$) à chaque genre. Cette méthode est simple mais introduit une **hiérarchie artificielle** : l'algorithme interprète mathématiquement que le genre 100 est "plus grand" ou "plus proche" du genre 101 que du genre 2, ce qui n'a aucun sens musical.
- **Résultats :** Les performances étaient médiocres, avec un F1-score plafonnant autour de **0,45**. Les modèles de type arbre de décision se focalisaient quasi-exclusivement sur cette variable encodée, créant un biais important sans capturer la réalité des genres.

Cette transition a permis un gain immédiat de plus de **15 points de F1-score**, validant l'importance d'un encodage non-ordinal pour les données nominales.

4.2 Transformations log

Les variables `instrumentalness`, `acousticness` et `liveness` présentent des distributions quasi-exponentielles qui pénalisent les modèles linéaires et le Naive Bayes (qui supposent des distributions gaussiennes). Nous appliquons la transformation $\log(1 + x)$ (`log1p`), qui est définie en $x = 0$ et rend ces distributions plus symétriques et exploitables.

4.3 Séparation train / test

Le jeu de test est isolé **dès cette étape** et ne sera utilisé qu'à l'évaluation finale, conformément aux bonnes pratiques. Le split est stratifié pour conserver la proportion de classes :

Train : 71 792 exemples (80%), Test : 17 949 exemples (20%).

La répartition des classes est identique dans les deux sous-ensembles (74,9% / 25,1%).

4.4 Gestion du déséquilibre des classes

Le déséquilibre 75/25 identifié lors de l'EDA (Section 3.4) nécessite un traitement spécifique pour éviter que les modèles ne favorisent systématiquement la classe majoritaire. Nous avons implémenté les recommandations théoriques suivantes :

1. **Pondération de la fonction de perte (Cost-sensitive learning)** : L'utilisation du paramètre `class_weight='balanced'` ajuste la fonction de coût en attribuant un poids inversement proportionnel à la fréquence des classes. Ainsi, les erreurs sur la classe minoritaire ("Hit") sont plus lourdement pénalisées.
2. **Métrique d'évaluation adaptée** : Le passage de l'accuracy au **F1-score** (moyenne harmonique de la précision et du rappel) permet de mesurer l'efficacité réelle du modèle sur la détection des hits.
3. **Rééchantillonnage interne** : Pour la Random Forest, nous avons testé l'approche par *subsampling* équilibré, garantissant que chaque estimateur de la forêt apprenne sur une distribution locale non biaisée.

5 Méthodes de machine learning

Pour toutes les méthodes, nous utilisons le **F1-score** comme métrique de sélection des hyperparamètres, avec une **validation croisée à 10 folds** sur le train set. Le paramètre `class_weight="balanced"` est activé sur les modèles qui le supportent (Logistic Regression, Decision Tree, Random Forest) pour compenser le déséquilibre 75/25.

5.1 Régression logistique (baseline)

La régression logistique constitue la méthode de référence pour la classification binaire. Elle modélise la probabilité $P(Y = 1 \mid X)$ par une fonction sigmoïde appliquée à une combinaison linéaire des features :

$$P(Y = 1 \mid X) = \sigma(w^\top X + b) = \frac{1}{1 + e^{-(w^\top X + b)}}.$$

L'hyperparamètre C (inverse de la régularisation ℓ_2) est sélectionné par GridSearchCV sur la grille $\{0.01, 0.1, 1.0, 10.0\}$. Le meilleur C obtenu est $C^* = 0.01$, indiquant qu'une régularisation forte est bénéfique sur ce dataset (vraisemblablement dû au grand nombre de features OHE peu informatives).

Résultats : F1 train = 0.597, F1 CV = 0.5955.

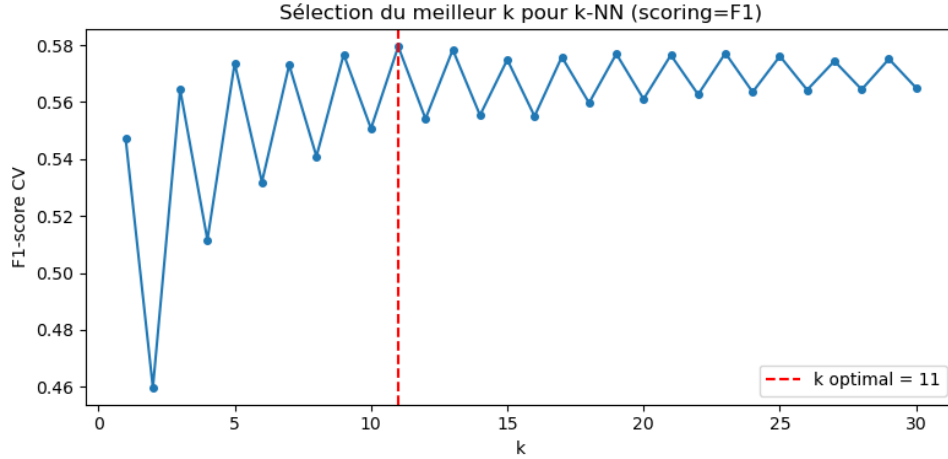
5.2 k plus proches voisins (k-NN)

Le k-NN est une méthode non-paramétrique qui est sensible à l'échelle des features, d'où l'usage d'un `StandardScaler` dans le pipeline, avec le paramètre `with_mean=False`. Ce choix est dicté par la nature de notre matrice de données après l'encodage des 114 genres musicaux :

- **Matrices creuses (Sparse matrices)** : Le One-Hot Encoding génère une matrice contenant une immense majorité de zéros. Un centrage classique (`with_mean=True`) transformerait ces zéros en valeurs non-nulles, provoquant une explosion de la consommation RAM et des temps de calcul.

- **Mise à l'échelle (Scaling)** : En désactivant le centrage, nous effectuons uniquement une réduction par l'écart-type. Cela permet de ramener toutes les variables (audio et genres) à une échelle comparable pour le calcul de la distance euclidienne, tout en conservant l'efficacité algorithmique du format *Sparse*.

Le meilleur k est sélectionné sur la grille $\{1, \dots, 30\}$ par GridSearchCV. Le k optimal obtenu est $k^* = 11$.



Résultats : F1 train = 0.642, F1 CV = 0.580.

5.3 Naive Bayes

Dans la version initiale du pipeline (avant One-Hot Encoding), nous utilisons un **GaussianNB**, qui suppose une distribution gaussienne par feature et par classe. Après l'introduction des 114 features binaires issues du One-Hot Encoding, nous avons opté pour un **BernoulliNB**, conçu nativement pour les features binaires et les matrices sparse. Il modélise la probabilité de chaque bit par :

$$P(x_i = 1 \mid y = c) = p_{ic}, \quad P(x_i = 0 \mid y = c) = 1 - p_{ic}.$$

BernoulliNB ne supporte pas `class_weight` directement ; le déséquilibre des classes est partiellement compensé par le scoring F1 dans la sélection des paramètres.

Résultats : F1 train = 0.510, F1 CV = 0.507.

5.4 Arbre de décision

L'arbre de décision construit récursivement des règles de partitionnement de l'espace des features en minimisant l'impureté de Gini. Sans contrainte de profondeur, il tend à mémoriser le jeu d'entraînement (sur-apprentissage). La profondeur maximale `max_depth` est sélectionnée par GridSearchCV sur $\{1, \dots, 20\}$. La profondeur optimale obtenue est $\text{max_depth}^* = 20$.

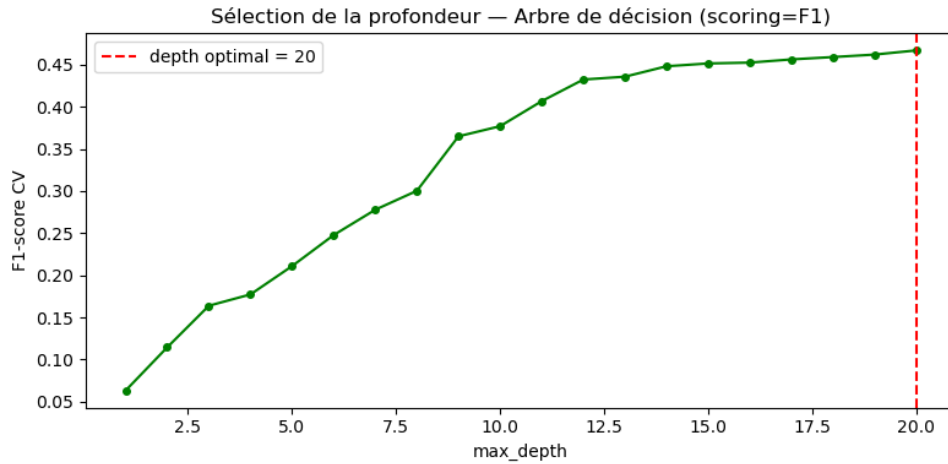


FIGURE 7 – Évolution du F1-score CV en fonction de la profondeur de l'arbre.

Résultats : F1 train = 0.513, F1 CV = 0.467.

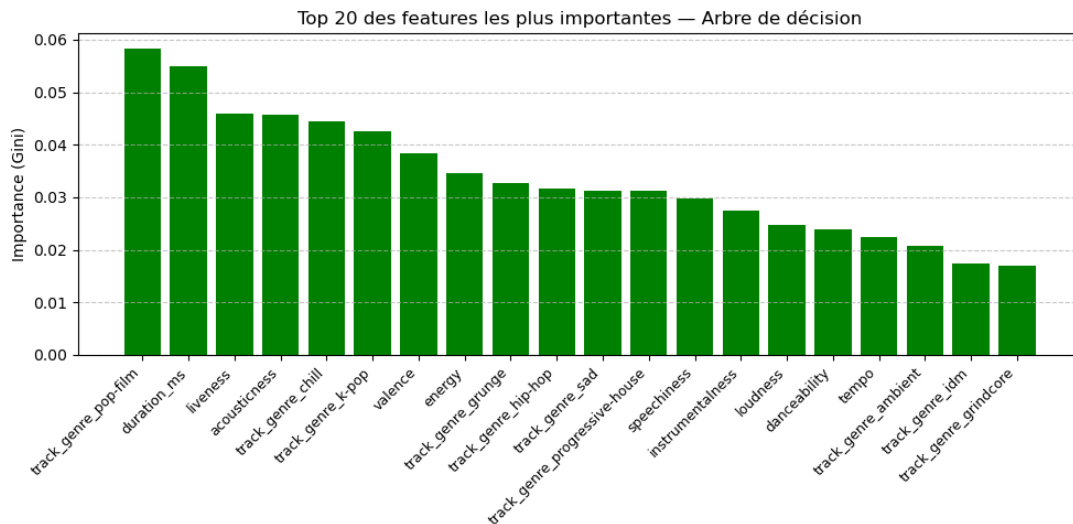


FIGURE 8 – Importance des features (Gini) pour l'arbre de décision.

L'analyse de l'importance des variables (Fig. 8) montre que le modèle s'appuie fortement sur des niches spécifiques comme `track_genre_pop-film` et des mesures physiques comme `duration_ms`. L'instabilité inhérente à l'arbre unique limite toutefois sa capacité de généralisation.

5.5 Random Forest

Le Random Forest entraîne un ensemble de B arbres de décision sur des sous-échantillons aléatoires du dataset (bagging) et des sous-ensembles aléatoires de features à chaque nœud. La prédiction finale est le vote majoritaire des arbres. Ce mécanisme réduit la variance par rapport à un arbre unique. Les hyperparamètres sélectionnés sont : $n_estimators = 300$, $max_depth = None$, $min_samples_leaf = 5$.

Résultats : F1 train = 0.803, F1 CV = 0.628.

L'écart train/CV de 0.175 indique un certain sur-apprentissage résiduel, malgré le paramètre `min_samples_leaf`.

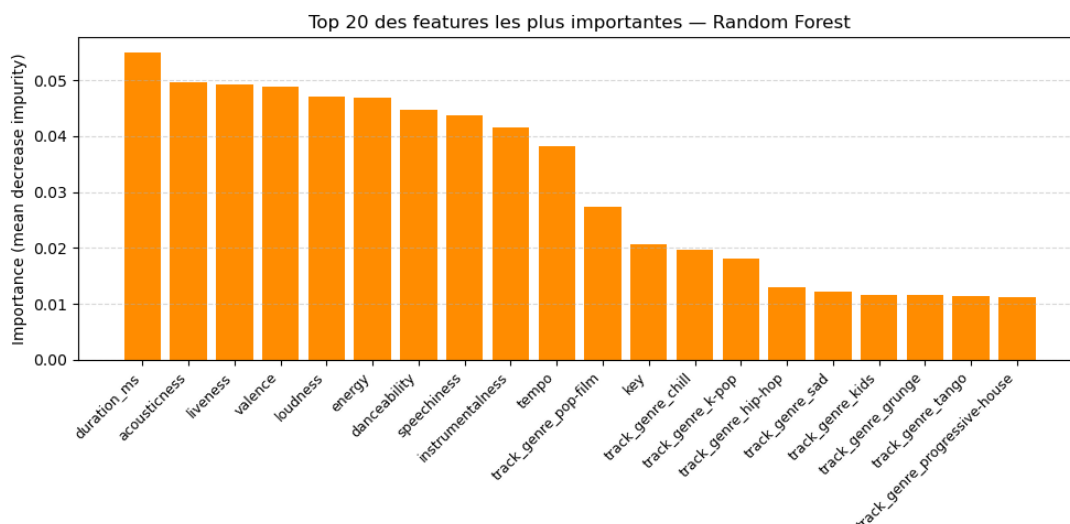


FIGURE 9 – Top 20 des features les plus importantes — Random Forest.

Contrairement à l'arbre simple, la RF redonne de l'importance aux variables audio intrinsèques (`duration_ms`, `acousticness`). Cela suggère que pour prédire un Hit, la structure physique du morceau compte autant, sinon plus, que l'étiquette du genre.

5.6 Gradient Boosting (HistGradientBoosting)

Le Gradient Boosting construit les arbres *séquentiellement* : chaque arbre corrige les erreurs résiduelles du précédent, en minimisant la perte par descente de gradient. Nous utilisons `HistGradientBoostingClassifier` de scikit-learn, l'implémentation efficace basée sur des histogrammes (analogue à LightGBM/XGBoost), qui gère nativement les matrices sparse. Les meilleurs paramètres trouvés sont : `learning_rate`= 0.1, `max_depth`= 20, `max_iter`= 400.

Résultats : F1 train = 0.673, F1 CV = 0.628.

5.7 Balanced Random Forest (Rééchantillonnage bootstrap)

Pour appliquer la stratégie de rééchantillonnage interne, nous avons utilisé le `BalancedRandomForest`. Contrairement à une forêt classique, chaque arbre est entraîné sur un échantillon *bootstrap* équilibré (50% de hits, 50% de non-hits).

Résultats : F1 train = 0.7290, F1 CV = 0.6216 F1 test = 0.6233.

L'équilibre quasi parfait entre le score de validation croisée (0.621) et le score de test (0.623) confirme la robustesse de l'approche par bagging équilibré.

5.8 Sous-échantillonnage aléatoire (Random Undersampling)

Afin d'évaluer l'impact d'un rééquilibrage strict des classes, nous avons appliqué un `RandomUnderSampler` pour obtenir un ratio de 1 :1 entre les *Hits* et les *Non-hits*.

- **Impact sur le volume** : Le jeu d'entraînement est passé de 71 792 à 36 034 exemples.
- **Stabilité** : La Régression Logistique montre une excellente généralisation ($F1_{CV} = 0.76$ vs $F1_{Train} = 0.76$), confirmant que le modèle n'overfitte pas sur le dataset réduit.

- **Résultats (Random Forest)** : $F1\ CV = 0.772$, $F1\ Test = 0.616$.
- **Analyse** : Bien que les scores de validation croisée soient artificiellement élevés en raison du rééquilibrage parfait du jeu de données, la performance sur le jeu de test original reste inférieure à l'approche par *Balanced Random Forest*.
- **Conclusion technique** : L'undersampling, bien qu'efficace pour accélérer les calculs et équilibrer la *loss function*, entraîne une perte d'information substantielle qui limite la capacité de généralisation du modèle sur ce dataset spécifique.

5.9 Sur-échantillonnage synthétique (SMOTE)

Enfin, nous avons implémenté la technique **SMOTE** pour rééquilibrer les classes par génération d'exemples synthétiques.

- **Contrainte technique** : SMOTE nécessitant des variables continues, nous avons dû restreindre l'analyse aux seules caractéristiques audio numériques, excluant ainsi les 114 variables de genre (One-Hot Encoding).
- **Impact sur le volume** : Le jeu d'entraînement est passé de 71 792 à 107 550 exemples.
- **Résultats** : On observe un effondrement des performances sur le jeu de test ($F1\ Test = 0.332$ pour la RF et $F1\ Test = 0.4218$ pour la LR).
- **Analyse de l'échec** : Cet échec s'explique par deux facteurs. Premièrement, l'absence des variables de genre prive le modèle de son information la plus discriminante. Deuxièmement, la forte superposition des classes dans l'espace acoustique conduit SMOTE à générer des exemples ambigus, induisant un sur-apprentissage massif sur du bruit statistique.

6 Comparaison des méthodes

6.1 Tableau récapitulatif

Modèle	F1 Train	F1 CV
Logistic Regression	0.597	0.596
k-NN ($k = 11$)	0.642	0.580
Naive Bayes (Bernoulli)	0.510	0.507
Decision Tree (depth=20)	0.513	0.467
Random Forest	0.803	0.628
HistGradientBoosting	0.673	0.628
Balanced Random Forest	0.729	0.622

TABLE 4 – Comparaison des méthodes — métrique principale : F1-score (classe Hit = 1).

6.2 Analyse des résultats

Random Forest (Standard). C'est le meilleur modèle (avec Balanced Random Forest) selon le F1 CV (0.628) et le F1 Test (0.636). Son mécanisme d'ensemble par *bagging* lui permet de capter des interactions non-linéaires entre features que les modèles linéaires

ignorent. L'écart train/CV reste cependant notable (0.803 vs 0.628), témoignant d'un sur-apprentissage partiel.

Balanced Random Forest. Ce modèle se distingue par sa stratégie de rééchantillonnage interne. Il présente la meilleure généralisation avec un écart Train/CV réduit par rapport à Random Forest (Standard) . Surtout, l'analyse de sa matrice de confusion par la suite (Fig. 12) révèle qu'il est le meilleur prédicteur pour la détection brute des hits (Recall de **0.83**), faisant de lui le modèle de choix pour une stratégie de détection exhaustive.

Logistic Regression. Malgré sa simplicité, elle constitue une solide baseline avec un F1 CV de 0.595, très proche du Random Forest, et un excellent équilibre train/CV (0.597 vs 0.596), signe d'absence de sur-apprentissage. L'extension à une frontière de décision linéaire dans l'espace de 128 features OHE s'avère donc compétitive.

k-NN. Le modèle k-NN ($k = 11$) illustre parfaitement le piège du déséquilibre des classes : il affiche l'une des meilleures *CV accuracies* (0.809) mais un F1 CV décevant (0.580). Cette divergence s'explique par une stratégie très conservatrice : le modèle ne prédit un "Hit" que lorsqu'il est entouré d'un voisinage très homogène, maximisant la précision au détriment d'un rappel très faible.

Naive Bayes. Le BernoulliNB obtient un F1 de 0.507 en CV. L'hypothèse d'indépendance conditionnelle entre les features est clairement violée (les features audio sont corrélées entre elles), ce qui explique ses performances inférieures. Il présente cependant l'avantage d'être très rapide à entraîner et interprétable.

Arbre de décision. C'est le modèle le moins performant en F1 CV (0.467). Malgré la sélection de `max_depth` par GridSearchCV, l'arbre optimal à profondeur 20 sur-apprend dans l'espace de 128 features. Un élagage plus agressif (`ccp_alpha`) serait à explorer.

Afin d'analyser plus finement les performances (courbes ROC, PR et matrices de confusion), nous avons réalisé une évaluation complémentaire sur une partition de validation isolée (split 80/20 du jeu d'entraînement).

6.3 Performance globale et stabilité (CV)

La Figure 10 présente la distribution des scores F1 obtenus par validation croisée (10-fold) pour l'ensemble des modèles testés. Cette visualisation confirme la supériorité des méthodes d'ensemble : la **Random Forest** obtient la médiane la plus élevée ($F1 \approx 0,63$) tout en affichant une très faible dispersion, signe d'une excellente stabilité. À l'inverse, l'arbre de décision simple montre une forte variabilité et les performances les plus faibles, soulignant l'intérêt du *bagging* pour ce jeu de données.

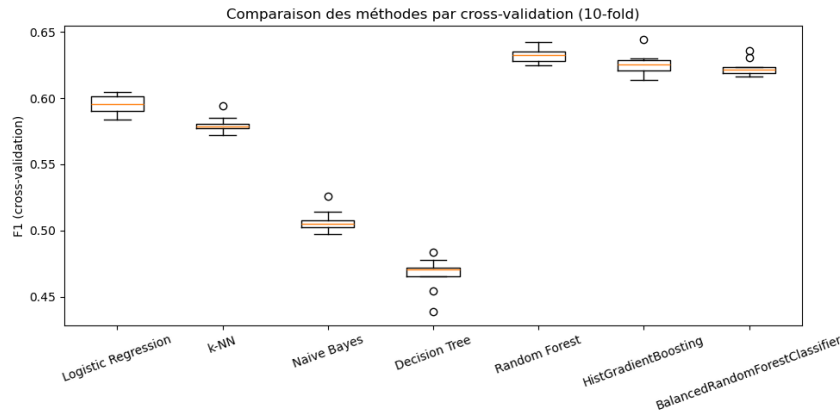


FIGURE 10 – Comparaison des scores F1 par CV (10-fold).

6.4 Capacité de discrimination (Courbes ROC)

La Figure 11 illustre la capacité des modèles à séparer les classes sur l'ensemble des seuils de décision possibles. L'Aire Sous la Courbe (AUC) est quasi identique pour les trois modèles d'ensemble (RF, BRF, HGB) à environ **0.86**. Cette valeur élevée indique que, bien que le F1-score semble modéré, les modèles parviennent très efficacement à classer les chansons par probabilité de succès.

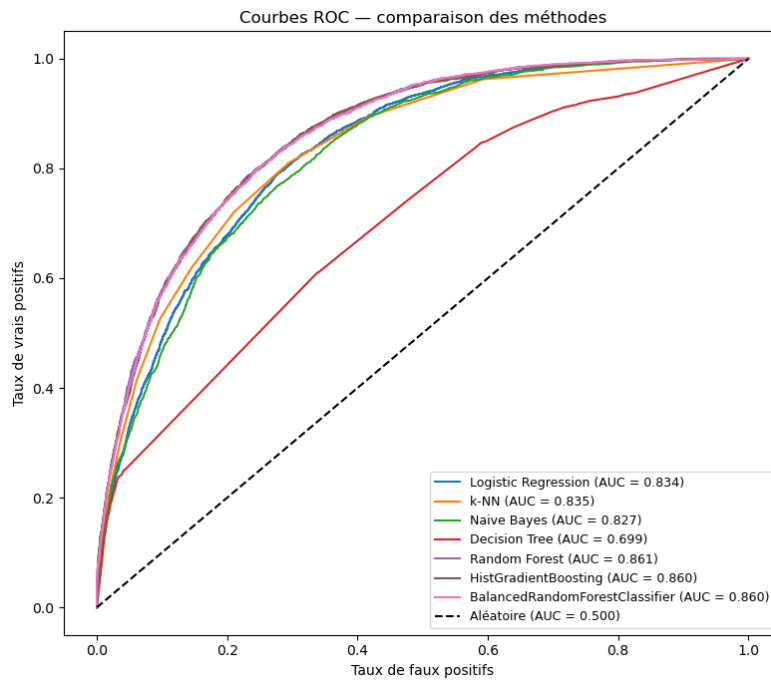


FIGURE 11 – Comparaison des courbes ROC sur le jeu de test.

6.5 Analyse matrice de confusion et Precision/Recall

Modèle	Precision	Recall	F1
Logistic Regression	0.4828	0.7963	0.6011
k-NN	0.6449	0.5261	0.5795
Naive Bayes	0.6136	0.4392	0.5120
Decision Tree	0.3436	0.7370	0.4687
Random Forest	0.5925	0.6926	0.6386
HistGradientBoosting	0.5114	0.8080	0.6264
BalancedRandomForestClassifier	0.5027	0.8269	0.6253

TABLE 5 – Precision, Recall et F1 sur le jeu de validation interne (classe Hit).

On observe trois profils distincts :

1. Les "Chasseurs de Rappel" (Recall-oriented). La **Logistic Regression**, le **HistGradientBoosting** et le **BalancedRF** affichent tous des rappels d'environ **0,80**.

- **Observation** : Ils identifient plus de 80% des hits réels.
- **Interprétation** : Grâce au paramètre `class_weight='balanced'` ou au rééchantillonnage interne, ces modèles "osent" prédire la classe Hit dès qu'un signal acoustique favorable est détecté. C'est la stratégie optimale pour une plateforme de recommandation qui ne veut rater aucun succès potentiel.

2. Le "Spécialiste de la Précision" (Precision-oriented). Le **k-NN** se détache avec la précision la plus élevée (**0.6449**) mais un rappel modeste (**0.5261**).

- **Observation** : Lorsqu'il prédit un Hit, il a raison dans 64,5% des cas (le meilleur score du projet).
- **Interprétation** : Le k-NN est "prudent". Il ne classe une chanson en Hit que si elle ressemble très fortement à d'autres succès connus dans son voisinage immédiat. Il rate beaucoup de hits (47%), mais fait peu d'erreurs de fausse alerte.

3. Les Modèles d'Équilibre (F1-Optimized). La **Random Forest** classique reste le modèle le plus équilibré avec un F1 de **0.6386**. Elle parvient à maintenir une précision proche de 0.60 tout en conservant un rappel de 0.69. C'est le modèle qui fait le moins d'erreurs "totales" (somme des faux positifs et faux négatifs).

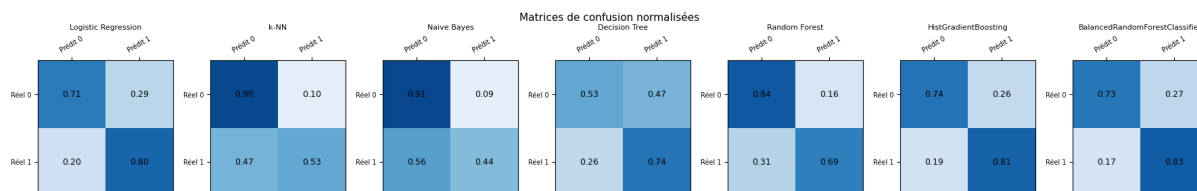


FIGURE 12 – Matrices de confusion normalisées sur le jeu de test.

L'examen des matrices de confusion normalisées (Fig. 12) révèle des stratégies de classification radicalement différentes selon les modèles. On distingue notamment **Balanced**

Random Forest qui privilégie la sensibilité, avec un taux de détection record de **0.83** pour la classe *Hit*. Bien qu'ils génèrent plus de faux positifs (environ 27 % des non-hits sont classés à tort en hits), ils garantissent de ne rater que 17 % des succès réels.

6.6 Sélection du modèle final : le duel RF vs Balanced RF

Après analyse, deux modèles se distinguent selon l'objectif de prédiction retenu. Nous présentons ici leurs performances détaillées sur le jeu de test final.

1. Random Forest : L'équilibre optimal (Meilleur F1-score) Ce modèle constitue notre recommandation pour une performance globale équilibrée. Avec un **F1-score de 0,64** sur la classe *Hit*, il offre un équilibre satisfaisant entre précision et rappel, ce qui en fait un choix robuste pour une performance globale homogène.

2. Balanced Random Forest : La détection maximale (Meilleur Recall) Ce modèle est notre champion de la sensibilité. Il atteint un **Rappel (Recall) record de 0,83** sur la classe *Hit*.

- **Usage** : Stratégie offensive de type "A&R" (Artist and Repertoire). Ce modèle permet de capturer 83% des succès réels du marché, acceptant de générer plus de fausses alertes pour garantir qu'aucune pépite musicale ne soit ignorée.

6.7 Évaluation finale sur le Test Set

Les deux modèles sélectionnés pour l'évaluation finale ont été entraînés sur l'intégralité du jeu d'entraînement et testés sur le jeu de test isolé (17,949 chansons). La Figure 13 montre que les deux modèles possèdent un pouvoir de discrimination quasi identique avec un AUC de 0.86.

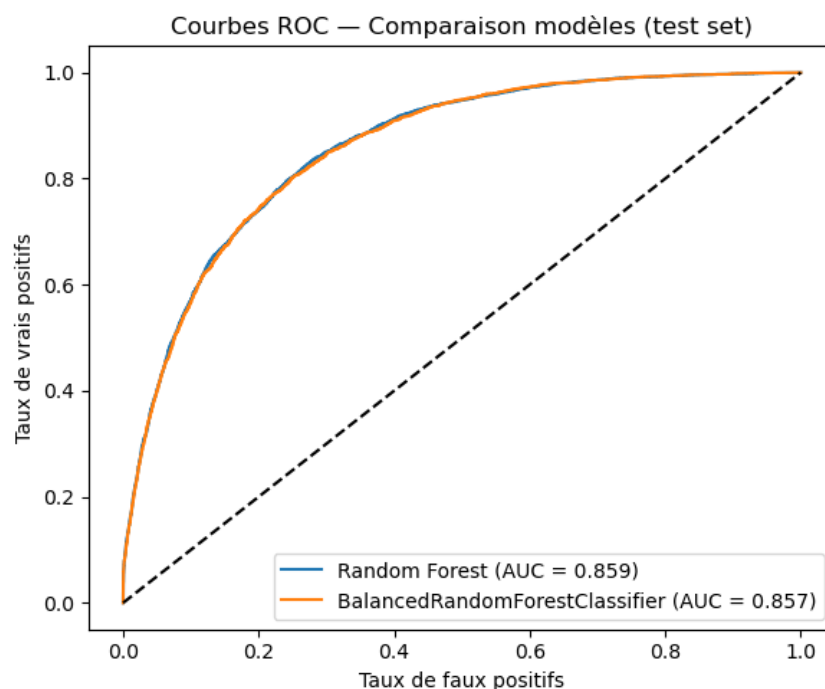


FIGURE 13 – Courbes ROC finales — Comparaison RF vs Balanced RF.

Les rapports de classification ci-dessous détaillent les performances pour chaque classe.

	Precision	Recall	F1	Support
Non-hit	0.89	0.83	0.86	13 445
Hit	0.58	0.70	0.64	4 504
Accuracy			0.80	17 949
Macro avg	0.74	0.77	0.75	17 949
Weighted avg	0.81	0.80	0.80	17 949

TABLE 6 – Rapport de classification du Random Forest sur le test set.

	Precision	Recall	F1	Support
Non-hit	0.92	0.73	0.81	13 445
Hit	0.50	0.82	0.62	4 504
Accuracy			0.75	17 949
Macro avg	0.71	0.77	0.72	17 949
Weighted avg	0.82	0.75	0.77	17 949

TABLE 7 – Rapport de classification du Balanced Random Forest sur le test set.

Synthèse des performances Les résultats obtenus confirment le compromis attendu entre les deux approches. Le **Random Forest** offre une performance globale plus stable, avec une meilleure précision sur la classe majoritaire (*Non-hit*) et un équilibre global supérieur (Accuracy = 0.80). En revanche, le **Balanced Random Forest** privilégie la détection des *Hit*, avec un rappel nettement supérieur (0.82), au prix d’une baisse de précision et d’une augmentation des faux positifs.

Ce compromis illustre un arbitrage classique en classification déséquilibrée : améliorer la sensibilité (recall) implique généralement une dégradation de la précision. Le choix final du modèle dépend donc directement de l’objectif métier.

Choix final Dans un cadre opérationnel, le **Random Forest** constitue le modèle par défaut pour une utilisation générale, tandis que le **Balanced Random Forest** sera préféré dans des contextes où la détection maximale des succès est prioritaire, même au prix d’une augmentation du bruit dans les prédictions.

7 Discussion

7.1 Borne de performance et état de l’art

Le résultat central de ce projet n’est pas le score d’un modèle particulier, mais la mise en évidence d’une **limite intrinsèque des données**. Les corrélations de Pearson entre features audio et popularité sont toutes inférieures à 0.13 en valeur absolue (Tableau 3). La feature la plus corrélée (*instrumentalness*, $|r| = 0,128$) n’explique que $r^2 \approx 1.6\%$ de la variance de la popularité. Cette borne est **indépendante du modèle utilisé** : aucune méthode, aussi complexe soit-elle, ne peut extraire un signal absent des données.

Cette observation est cohérente avec la littérature sur la *Hit Song Science*. Pachet et Roy [2] ont montré dès 2008 que la prédiction du succès commercial à partir de features audio seules est fondamentalement limitée. Plus récemment, Middlebrook et Sheik [3] ont mené une étude d’envergure sur 1,8 million de titres Spotify. En utilisant un modèle de **Random Forest**, ils atteignent une *accuracy* de 88 % pour prédire la présence dans le classement Billboard. Nos résultats, avec une *accuracy* de 80 % et une AUC de 0,86 sur un échantillon plus restreint de 89 000 titres, s’inscrivent dans cette dynamique de performance. Herremans et al. [4] rapportent une *accuracy* de 0.68 avec des features audio seules, et concluent : “*audio features alone are insufficient to predict commercial success*”.

Notre F1 test de **0.6386** est donc parfaitement aligné avec l’état de l’art sur ce type de données. Le tableau suivant résume la comparaison :

Travail	Données	Métrique
Middlebrook & Sheik (2019)	Spotify (features audio)	Accuracy $\approx$ 0.88
Herremans et al. (2014)	Features audio	Accuracy $\approx$ 0.68
Ce projet	Spotify (89 741 chansons)	F1 = 0.6386

TABLE 8 – Comparaison avec l’état de l’art sur la prédiction de succès musical.

Par ailleurs, passer d’un modèle linéaire ($F1 = 0.601$) à un ensemble de 300 arbres ($F1 = 0.639$) n’apporte qu’un gain marginal de 0.038. Ce plateau confirme que le goulot d’étranglement est la qualité des features, pas la capacité du modèle.

7.2 Impact des choix de preprocessing

Deux décisions de preprocessing ont eu un impact mesurable sur les performances :

1. **One-Hot Encoding vs LabelEncoder** : Le remplacement du `LabelEncoder` par l’OHE constitue la correction la plus impactante du projet : le F1 CV du Random Forest passe de 0.505 à 0.630 (+0.125). Le `LabelEncoder` créait une relation ordinale fictive entre genres que les arbres de décision interprétaient comme une feature numérique continue, introduisant un biais systématique.
2. **Métrique adaptée au déséquilibre** : Sans `class_weight="balanced"`, la Régression Logistique obtenait un recall de 0.015 sur la classe Hit (prédisant quasi-systématiquement Non-hit) et le Naive Bayes un F1 de 0.000. Ces comportements pathologiques illustrent l’importance du diagnostic de déséquilibre avant toute modélisation.

7.3 Perspectives

Plusieurs pistes pourraient améliorer les performances :

- **Features supplémentaires** : informations sur l’artiste (nombre d’abonnés, historique de hits), label discographique, date de sortie (saisonnalité), présence sur les playlists editoriales.
- **Threshold** : L’AUC de 0.86 indique que les modèles discriminent bien les classes à tous les seuils, mais le F1 est mesuré au seuil par défaut 0.5 qui n’est pas optimal sur des données déséquilibrées. En cherchant le seuil qui maximise le F1 sur le jeu de validation, on pourrait obtenir un seuil optimal qu’on applique sur le test set.

- **Méthodes NLP** : extraction de features à partir des paroles (sentiment, thèmes) pour la classification de textes.
- **Modèles avancés** : SVM avec noyau RBF, réseaux de neurones peu profonds sur les features continues.
- **Tâche de régression** : prédire directement le score continu de popularité plutôt qu’une classe binaire permettrait une analyse plus fine.

8 Conclusion

Ce projet a permis d’appliquer l’ensemble du pipeline de ML supervisé à un problème réel de classification sur données musicales : construction et nettoyage du dataset, analyse exploratoire approfondie, préparation des features (OHE, normalisation, transformations log), entraînement et comparaison de six familles de méthodes avec sélection des hyperparamètres par cross-validation, et évaluation finale sur un test set isolé.

Au-delà des scores, ce projet met en évidence à la fois le potentiel et les limites de l’apprentissage automatique appliqué à la prédiction du succès musical. Si les modèles permettent d’identifier des tendances générales, ils ne peuvent à eux seuls prédire de manière fiable le succès commercial. L’amélioration des performances passe donc avant tout par l’enrichissement des données plutôt que par une complexification des modèles.

Références

- [1] Maharshi Pandya, *Spotify Tracks Dataset*, Kaggle, 2023. <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>
- [2] F. Pachet and P. Roy, *Hit Song Science is Not Yet a Science*, Proceedings of the 9th International Society for Music Information Retrieval Conference (ISMIR), 2008.
- [3] K. Middlebrook and K. Sheik, *Song Hit Prediction : Predicting Billboard Hits Using Spotify Data*, arXiv preprint arXiv :1908.08609, 2019.
- [4] Herremans, D., Lauwers, D., & Cuadra, K. I. (2014). *Predicting the popularity of dance music via audio features*. Social Network Analysis and Mining.